

SHA-2

SHA-2 (Secure Hash Algorithm 2) is a set of [cryptographic hash functions](#) designed by the United States National Security Agency (NSA) and first published in 2001.^{[3][4]} They are built using the [Merkle–Damgård construction](#), from a one-way compression function itself built using the [Davies–Meyer structure](#) from a specialized block cipher.

SHA-2 includes significant changes from its predecessor, **SHA-1**. The SHA-2 family consists of six hash functions with digests (hash values) that are 224, 256, 384 or 512 bits.^[5] **SHA-224**, **SHA-256**, **SHA-384**, **SHA-512**, **SHA-512/224**, **SHA-512/256**. SHA-256 and SHA-512 are hash functions whose digests are eight 32-bit and 64-bit words, respectively. They use different shift amounts and additive constants, but their structures are otherwise virtually identical, differing only in the number of rounds. SHA-224 and SHA-384 are truncated versions of SHA-256 and SHA-512 respectively, computed with different initial values. SHA-512/224 and SHA-512/256 are also truncated versions of SHA-512, but the initial values are generated using the method described in [Federal Information Processing Standards](#) (FIPS) PUB 180-4.

SHA-2 was first published by the [National Institute of Standards and Technology](#) (NIST) as a U.S. federal standard. The SHA-2 family of algorithms are patented in the U.S.^[6] The United States has released the patent under a [royalty-free](#) license.^[5]

As of 2024, the **SHA-2** cryptographic hash function remains secure against known attacks, including those from [quantum computers](#).^[7]

Hash standard

With the publication of FIPS PUB 180-2, NIST added three additional hash functions in the SHA family. The algorithms are collectively known as SHA-2, named after their digest lengths (in bits): SHA-256, SHA-384, and SHA-512.

The algorithms were first published in 2001 in the draft FIPS PUB 180-2, at which time public review and comments were accepted. In August 2002, FIPS PUB 180-2 became the new [Secure Hash Standard](#), replacing FIPS PUB 180-1, which was released in April 1995. The updated standard included the original SHA-1 algorithm, with updated technical notation consistent with that describing the inner workings of the SHA-2 family.^[4]

In February 2004, a change notice was published for FIPS PUB 180-2, specifying an additional variant, SHA-224, defined to match the key length of two-key Triple DES.^[8] In October 2008, the standard was updated in FIPS PUB 180-3, including SHA-224 from the change notice, but otherwise making no fundamental changes to the standard. The primary motivation for updating the standard was relocating security information about the hash algorithms and recommendations for their use to Special Publications 800-107 and 800-57.^{[9][10][11]} Detailed test data and example message digests were also removed from the standard, and provided as separate documents.^[12]

In January 2011, NIST published SP800-131A, which specified a move from the then-current minimum of 80-bit security (provided by SHA-1) allowable for federal government use until the end of 2013, to 112-bit security (provided by SHA-2) being both the minimum requirement (starting in 2014) and the recommended [security level](#) (starting from the publication date in 2011).^[13]

In March 2012, the standard was updated in FIPS PUB 180-4, adding the hash functions SHA-512/224 and SHA-512/256, and describing a method for generating initial values for truncated versions of SHA-512. Additionally, a restriction on padding the input data prior to hash calculation was removed, allowing hash data to be calculated simultaneously with content generation, such as a real-time video or audio feed. Padding the final data block must still occur prior to hash output.^[14]

In July 2012, NIST revised SP800-57, which provides guidance for cryptographic key management. The publication disallowed creation of digital signatures with a hash security lower than 112 bits after 2013. The previous revision from 2007 specified the cutoff to be the end of 2010.^[11] In August 2012, NIST revised SP800-107 in the same manner.^[10]

In March 2023, NIST announced its decision to revise FIPS 180-4.^[15] FIPS 180-5 will remove the SHA-1 specification, add guidance from SP 800-107, and include editorial updates.

SHA-2

The diagram illustrates the SHA-2 architecture. It shows two rows of input blocks labeled A through H. The top row of blocks is connected to a series of processing blocks labeled Ch, K1, and M2. The bottom row of blocks is connected to a series of processing blocks labeled A, B, C, D, E, F, G, and H. The output of the processing blocks is shown as a single block labeled K.

General	
Designers	National Security Agency
First published	2001
Series	(SHA-0) , SHA-1 , SHA-2 , SHA-3
Certification	FIPS PUB 180-4 , CRYPTREC , NESSIE
Detail	
Digest sizes	224, 256, 384, or 512 bits
Structure	Merkle–Damgård construction with Davies–Meyer compression function
Rounds	64 or 80
Best public cryptanalysis	
A 2011 attack breaks preimage resistance for 57 out of 80 rounds of SHA-512, and 52 out of 64 rounds for SHA-256. ^[1] Pseudo-collision attack against up to 46 rounds of SHA-256. ^[2]	

Secure Hash Algorithms

Concepts

[hash functions](#), [SHA](#), [DSA](#)

Main standards

[SHA-0](#), [SHA-1](#), [SHA-2](#), [SHA-3](#)

The [NIST hash function competition](#) selected a new hash function, [SHA-3](#), in 2012.^[16] The SHA-3 algorithm is not derived from SHA-2.

Applications

The SHA-2 hash function is implemented in some widely used security applications and protocols, including [TLS](#) and [SSL](#), [PGP](#), [SSH](#), [S/MIME](#), and [IPsec](#). The inherent computational demand of SHA-2 algorithms has driven the proposal of more efficient solutions, such as those based on application-specific integrated circuits (ASICs) hardware accelerators.^[17]

SHA-256 is used for authenticating [Debian](#) software packages^[18] and in the [DKIM](#) message signing standard; SHA-512 is part of a system to authenticate archival video from the [International Criminal Tribunal of the Rwandan genocide](#).^[19] SHA-256 and SHA-512 are used in [DNSSEC](#).^[20] Linux distributions usually use 512-bit SHA-2 for secure password hashing.^{[21][22]}

Several [cryptocurrencies](#), including [Bitcoin](#), use SHA-256 for verifying transactions and calculating [proof of work](#)^[23] or [proof of stake](#).^[24] The rise of [ASIC](#) SHA-2 accelerator chips has led to the use of [scrypt](#)-based proof-of-work schemes.

In both 4G and 5G mobile networks, HMAC-SHA-256 is utilized as a key derivation function (KDF) to generate cryptographic keys essential for securing communications. This process is defined in the 3rd Generation Partnership Project (3GPP) Technical Specifications TS 33.401^[25] and TS 33.501,^[26] which outline the security architecture and procedures for these networks.

SHA-1, SHA-2, and SHA-3 are the [Secure Hash Algorithms](#) required by law for use in certain [U.S. Government](#) applications, including use within other cryptographic algorithms and protocols, for the protection of sensitive unclassified information. FIPS PUB 180-1 also encouraged adoption and use of SHA-1 by private and commercial organizations. SHA-1 is being retired for most government uses; the U.S. National Institute of Standards and Technology says, "NIST recommends that federal agencies transition away from SHA-1 for all applications as soon as possible. Federal agencies should use SHA-2 or SHA-3 as an alternative to SHA-1."^[27] NIST's directive that U.S. government agencies ought to, but not explicitly must, stop uses of SHA-1 after 2010^[28] was hoped to accelerate migration away from SHA-1.

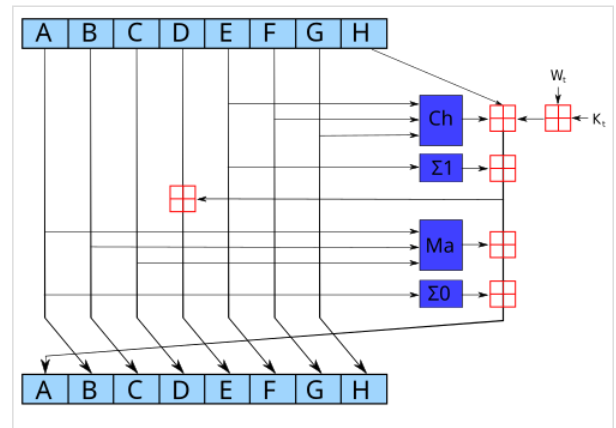
The SHA-2 functions were not quickly adopted initially, despite better security than SHA-1. Reasons might include lack of support for SHA-2 on systems running Windows XP SP2 or older^[29] and a lack of perceived urgency since SHA-1 collisions had not yet been found. The [Google Chrome](#) team announced a plan to make their web browser gradually stop honoring SHA-1-dependent TLS certificates over a period from late 2014 and early 2015.^{[30][31][32]} Similarly, Microsoft announced^[33] that [Internet Explorer](#) and [Edge \[Legacy\]](#) would stop honoring public SHA-1-signed TLS certificates from February 2017. Mozilla disabled SHA-1 in [Firefox](#) during early January 2016, but had to re-enable it temporarily via an update, after problems with web-based user interfaces of some router models and [security appliances](#).^[34]

Cryptanalysis and validation

For a hash function for which L is the number of bits in the [message digest](#), finding a message that corresponds to a given message digest can always be done using a [brute force](#) search in 2^L evaluations. This is called a [preimage attack](#) and may or may not be practical depending on L and the particular computing environment. The second criterion, finding two different messages that produce the same message digest, known as a [collision](#), requires on average only $2^{L/2}$ evaluations using a [birthday attack](#).

Some of the applications that use cryptographic hashes, such as password storage, are only minimally affected by a [collision attack](#). Constructing a password that works for a given account requires a preimage attack, as well as access to the hash of the original password (typically in the [shadow](#) file) which may or may not be trivial. Reversing password encryption (e.g., to obtain a password to try against a user's account elsewhere) is not made possible by the attacks. (However, even a secure password hash cannot prevent brute-force attacks on [weak passwords](#).)

In the case of document signing, an attacker could not simply fake a signature from an existing document—the attacker would have to produce a pair of documents, one innocuous and one damaging, and get the private key holder to sign the innocuous document. There are practical circumstances in which this is possible; until the end of 2008, it was possible to create forged [SSL](#) certificates using an [MD5](#) collision which would be accepted by widely used web browsers.^[35]



One iteration in a SHA-2 family compression function. The blue components perform the following operations:

$$\begin{aligned} \text{Ch}(E, F, G) &= (E \wedge F) \oplus (\neg E \wedge G) \\ \text{Ma}(A, B, C) &= (A \wedge B) \oplus (A \wedge C) \oplus (B \wedge C) \\ \Sigma_0(A) &= (A \gg 2) \oplus (A \gg 13) \oplus (A \gg 22) \\ \Sigma_1(E) &= (E \gg 6) \oplus (E \gg 11) \oplus (E \gg 25) \end{aligned}$$

The bitwise rotation uses different constants for SHA-512. The given numbers are for SHA-256.

The red ■ is addition modulo 2^{32} for SHA-256, or 2^{64} for SHA-512.

Increased interest in cryptographic hash analysis during the SHA-3 competition produced several new attacks on the SHA-2 family, the best of which are given in the table below. Only the collision attacks are of practical complexity; none of the attacks extend to the full round hash function.

At FSE 2012, researchers at Sony gave a presentation suggesting pseudo-collision attacks could be extended to 52 rounds on SHA-256 and 57 rounds on SHA-512 by building upon the [biclique](#) pseudo-preimage attack.^[36]

Published in	Year	Attack method	Attack	Variant	Rounds	Complexity
<i>New Collision Attacks Against Up To 24-step SHA-2</i> ^{[37][38]}	2008	Differential	Collision	SHA-256	24/64	$2^{15.5}$
				SHA-512	24/80	$2^{22.5}$
<i>Preimages for step-reduced SHA-2</i> ^[39]	2009	<u>Meet-in-the-middle</u>	Preimage	SHA-256	42/64	$2^{251.7}$
					43/64	$2^{254.9}$
				SHA-512	42/80	$2^{502.3}$
					46/80	$2^{511.5}$
<i>Advanced meet-in-the-middle preimage attacks</i> ^[40]	2010	<u>Meet-in-the-middle</u>	Preimage	SHA-256	42/64	$2^{248.4}$
				SHA-512	42/80	$2^{494.6}$
<i>Higher-Order Differential Attack on Reduced SHA-256</i> ^[2]	2011	<u>Differential</u>	Pseudo-collision	SHA-256	46/64	2^{178}
					33/64	2^{46}
<i>Bicliques for Preimages: Attacks on Skein-512 and the SHA-2 family</i> ^[1]	2011	<u>Biclique</u>	Preimage	SHA-256	45/64	$2^{255.5}$
				SHA-512	50/80	$2^{511.5}$
			Pseudo-preimage	SHA-256	52/64	2^{255}
				SHA-512	57/80	2^{511}
<i>Improving Local Collisions: New Attacks on Reduced SHA-256</i> ^[41]	2013	Differential	Collision	SHA-256	31/64	$2^{65.5}$
			Pseudo-collision	SHA-256	38/64	2^{37}
<i>Branching Heuristics in Differential Collision Search with Applications to SHA-512</i> ^[42]	2014	Heuristic differential	Pseudo-collision	SHA-512	38/80	$2^{40.5}$
<i>Analysis of SHA-512/224 and SHA-512/256</i> ^[43]	2016	Differential	Collision	SHA-256	28/64	practical
				SHA-512	27/80	practical
			Pseudo-collision	SHA-512	39/80	practical
<i>New Records in Collision Attacks on SHA-2</i> ^[44]	2024	Differential	Collision	SHA-256	31/64	$2^{49.8}$
				SHA-512	31/80	$2^{115.6}$
			Pseudo-collision	SHA-256	39/64	practical

Official validation

Implementations of all FIPS-approved security functions can be officially validated through the [CMVP program](#), jointly run by the [National Institute of Standards and Technology](#) (NIST) and the [Communications Security Establishment](#) (CSE). For informal verification, a package to generate a high number of test vectors is made available for download on the NIST site; the resulting verification, however, does not replace the formal CMVP validation, which is required by law^[45] for certain applications.

As of December 2013, there are over 1300 validated implementations of SHA-256 and over 900 of SHA-512, with only 5 of them being capable of handling messages with a length in bits not a multiple of eight while supporting both variants.^[46]

Test vectors

Hash values of an empty string (i.e., a zero-length input text).

```
SHA224("")
0x d14a028c2a3a2bc9476102bb288234c415a2b01f828ea62ac5b3e42f
SHA256("")
0x e3b0c44298fc1c149afb4c8996fb92427ae41e4649b934ca495991b7852b855
SHA384("")
0x 38b060a751ac96384cd9327eb1b1e36a21fdb71114be07434c0cc7bf63f6e1da274edebfe76f65fbd51ad2f14898b95b
SHA512("")
0x cf83e1357eefb8bdf1542850d66d8007d620e4050b5715dc83f4a921d36ce9ce47d0d13c5d85f2b0ff8318d2877eec2f63b931bd47417a81a538327af927da3e
SHA512/224("")
0x 6ed0dd02806fa89e25de060c19d3ac86cabb87d6a0ddd05c333b84f4
SHA512/256("")
0x c672b8d1ef56ed28ab87c3622c5114069bdd3ad7b8f9737498d0c01ecef0967a
```

Even a small change in the message will (with overwhelming probability) result in a different hash, due to the avalanche effect. For example, adding a period to the end of the following sentence changes approximately half (111 out of 224) of the bits in the hash, equivalent to picking a new hash at random:

```
SHA224("The quick brown fox jumps over the lazy dog")
0x 730e109bd7a8a32b1cb9d9a09aa2325d2430587ddbc0c38bad911525
SHA224("The quick brown fox jumps over the lazy dog.")
0x 619cba8e8e05826e9b8c519c0a5c68f4fb653e8a3d8aa04bb2c8cd4c
```

Pseudocode

Pseudocode for the SHA-256 algorithm follows. Note the great increase in mixing between bits of the $w[16..63]$ words compared to SHA-1.

Note 1: All variables are 32 bit unsigned integers and addition is calculated modulo 2^{32}
Note 2: For each round, there is one round constant $k[i]$ and one entry in the message schedule array $w[i]$, $0 \leq i \leq 63$
Note 3: The compression function uses 8 working variables, a through h
Note 4: Big-endian convention is used when expressing the constants in this pseudocode, and when parsing message block data from bytes to words, for example, the first word of the input message "abc" after padding is 0x61626380

Initialize hash values:

(first 32 bits of the fractional parts of the square roots of the first 8 primes 2..19):

```
h0 := 0x6a09e667
h1 := 0xbb67ae85
h2 := 0x3c6ef372
h3 := 0xa54ff53a
h4 := 0x510e527f
h5 := 0x9b05688c
h6 := 0x1f83d9ab
h7 := 0x5be0cd19
```

Initialize array of round constants:

(first 32 bits of the fractional parts of the cube roots of the first 64 primes 2..311):

```
k[0..63] :=
0x428a2f98, 0x71374491, 0xb5c0fbcf, 0xe9b5dba5, 0x3956c25b, 0x59f111f1, 0x923f82a4, 0xab1c5ed5,
0xd807aa98, 0x12835b01, 0x243185be, 0x550c7dc3, 0x72be5d74, 0x80deb1fe, 0x9bdc06a7, 0xc19bf174,
0xe49b69c1, 0xefbe4786, 0x0fc19dc6, 0x240ca1cc, 0x2de92cf6, 0x4a7484aa, 0x5cb0a9dc, 0x76f988da,
0x983e5152, 0xa831c66d, 0xb00327c8, 0xbf597fc7, 0xc6e00bf3, 0xd5a79147, 0x06ca6351, 0x14292967,
0x27b70a85, 0x2e1b2138, 0x4d2c6dfe, 0x53380d13, 0x650a7354, 0x766a0abb, 0x81c2c92e, 0x92722c85,
0xa2bfe8a1, 0xa81a664b, 0xc24b8b70, 0xc76c51a3, 0xd192e819, 0xd6990624, 0xf40e3585, 0x106aa070,
0x19a4c116, 0x1e376c08, 0x2748774c, 0x34b0bcb5, 0x391c0cb3, 0x4ed8aa4a, 0x5b9cca4f, 0x682e6ff3,
0x748f82ee, 0x78a5636f, 0x84c87814, 0x8cc70208, 0x90befffa, 0xa4506ceb, 0xbef9a3f7, 0xc67178f2
```

Pre-processing (Padding):

begin with the original message of length L bits

append a single '1' bit

append K '0' bits, where K is the minimum number ≥ 0 such that $(L + 1 + K + 64)$ is a multiple of 512

append L as a 64-bit big-endian integer, making the total post-processed length a multiple of 512 bits

such that the bits in the message are: (original message of length L) 1 (K zeros) (L as 64 bit integer) , (the number of bits will be a multiple of 512)

Process the message in successive 512-bit chunks:

break message into 512-bit chunks

for each chunk

create a 64-entry message schedule array $w[0..63]$ of 32-bit words

(The initial values in $w[0..63]$ don't matter, so many implementations zero them here)

copy chunk into first 16 words $w[0..15]$ of the message schedule array

Extend the first 16 words into the remaining 48 words $w[16..63]$ of the message schedule array:

for i from 16 to 63

$s0 := (w[i-15] \text{ rightrotate } 7) \text{ xor } (w[i-15] \text{ rightrotate } 18) \text{ xor } (w[i-15] \text{ rightshift } 3)$

$s1 := (w[i-2] \text{ rightrotate } 17) \text{ xor } (w[i-2] \text{ rightrotate } 19) \text{ xor } (w[i-2] \text{ rightshift } 10)$

$w[i] := w[i-16] + s0 + w[i-7] + s1$

Initialize working variables to current hash value:

```
a := h0
b := h1
c := h2
d := h3
e := h4
f := h5
g := h6
h := h7
```

Compression function main loop:

```
for i from 0 to 63
  S1 := (e rightrotate 6) xor (e rightrotate 11) xor (e rightrotate 25)
  ch := (e and f) xor ((not e) and g)
  temp1 := h + S1 + ch + k[i] + w[i]
  S0 := (a rightrotate 2) xor (a rightrotate 13) xor (a rightrotate 22)
  maj := (a and b) xor (a and c) xor (b and c)
  temp2 := S0 + maj

  h := g
  g := f
  f := e
  e := d + temp1
  d := c
  c := b
  b := a
  a := temp1 + temp2
```

Add the compressed chunk to the current hash value:

```
h0 := h0 + a
h1 := h1 + b
h2 := h2 + c
h3 := h3 + d
h4 := h4 + e
h5 := h5 + f
h6 := h6 + g
h7 := h7 + h
```

Produce the final hash value (big-endian):

```
digest := hash := h0 append h1 append h2 append h3 append h4 append h5 append h6 append h7
```

The computation of the `ch` and `maj` values can be optimized the same way as described for SHA-1.

SHA-224 is identical to SHA-256, except that:

- the initial hash values `h0` through `h7` are different, and
- the output is constructed by omitting `h7`.

SHA-224 initial hash values (in big endian):

(The second 32 bits of the fractional parts of the square roots of the 9th through 16th primes 23..53)

```
h[0..7] :=
  0xc1059ed8, 0x367cd507, 0x3070dd17, 0xf70e5939, 0xffc00b31, 0x68581511, 0x64f98fa7, 0xbefa4fa4
```

SHA-512 is identical in structure to SHA-256, but:

- the message is broken into 1024-bit chunks,
- the initial hash values and round constants are extended to 64 bits,
- there are 80 rounds instead of 64,
- the message schedule array `w` has 80 64-bit words instead of 64 32-bit words,
- to extend the message schedule array `w`, the loop is from 16 to 79 instead of from 16 to 63,
- the round constants are based on the first 80 primes 2..409,
- the word size used for calculations is 64 bits long,
- the appended length of the message (before pre-processing), in *bits*, is a 128-bit big-endian integer, and
- the shift and rotate amounts used are different.

SHA-512 initial hash values (in big-endian):

```
h[0..7] := 0x6a09e667f3bcc908, 0xbb67ae8584caa73b, 0x3c6ef372fe94f82b, 0xa54ff53a5f1d36f1,
           0x510e527fade682d1, 0x9b05688c2b3e6c1f, 0x1f83d9abfb41bd6b, 0x5be0cd19137e2179
```

SHA-512 round constants:

```
k[0..79] := 0x428a2f98d728ae22, 0x7137449123ef65cd, 0xb5c0fbcfec4d3b2f, 0xe9b5dba58189dbbc,
0x3956c25bf348b538,
           0x59f111f1b605d019, 0x923f82a4af194f9b, 0xab1c5ed5da6d8118, 0xd807aa98a3030242,
0x12835b0145706fbe,
           0x243185be4ee4b28c, 0x550c7dc3d5ffb4e2, 0x72be5d74f27b896f, 0x80deb1fe3b1696b1,
0x9bdc06a725c71235,
```

```

0xc19bf174cf692694, 0xe49b69c19ef14ad2, 0xefbe4786384f25e3, 0x0fc19dc68b8cd5b5,
0x240ca1cc77ac9c65,
0x2de92c6f592b0275, 0x4a7484aa6ea6e483, 0x5cb0a9dcdbd41fbd4, 0x76f988da831153b5,
0x983e5152ee66dfab,
0xa831c66d2db43210, 0xb00327c898fb213f, 0xbf597fc7beef0ee4, 0xc6e00bf33da88fc2,
0xd5a79147930aa725,
0x06ca6351e003826f, 0x142929670a0e6e70, 0x27b70a8546d22ffc, 0x2e1b21385c26c926,
0x4d2c6dfc5ac42aed,
0x53380d139d95b3df, 0x650a73548baf63de, 0x766a0abb3c77b2a8, 0x81c2c92e47edaee6,
0x92722c851482353b,
0xa2bfe8a14cf10364, 0xa81a664bbc423001, 0xc24b8b70d0f89791, 0xc76c51a30654be30,
0xd192e819d6ef5218,
0xd69906245565a910, 0xf40e35855771202a, 0x106aa07032bbd1b8, 0x19a4c116b8d2d0c8,
0x1e376c085141ab53,
0x2748774cdf8eeb99, 0x34b0bcb5e19b48a8, 0x391c0cb3c5c95a63, 0x4ed8aa4ae3418acb,
0x5b9cca4f7763e373,
0x682e6ff3d6b2b8a3, 0x748f82ee5defb2fc, 0x78a5636f43172f60, 0x84c87814a1f0ab72,
0x8cc702081a6439ec,
0x90befffa23631e28, 0xa4506cebd82bde9, 0xbef9a3f7b2c67915, 0xc67178f2e372532b,
0xca273ecdea26619c,
0xd186b8c721c0c207, 0xeda7dd6cde0eb1e, 0xf57d4f7fee6ed178, 0x06f067aa72176fba,
0x0a637dc5a2c898a6,
0x113f9804bef90dae, 0x1b710b35131c471b, 0x28db77f523047d84, 0x32caab7b40c72493,
0x3c9ebe0a15c9bebc,
0x431d67c49c100d4c, 0x4cc5d4becb3e42b6, 0x597f299cfc657e2a, 0x5fcb6fab3ad6faec,
0x6c44198c4a475817

```

SHA-512 Sum & Sigma:

```

S0 := (a rightrotate 28) xor (a rightrotate 34) xor (a rightrotate 39)
S1 := (e rightrotate 14) xor (e rightrotate 18) xor (e rightrotate 41)

```

```

s0 := (w[i-15] rightrotate 1) xor (w[i-15] rightrotate 8) xor (w[i-15] rightshift 7)
s1 := (w[i-2] rightrotate 19) xor (w[i-2] rightrotate 61) xor (w[i-2] rightshift 6)

```

SHA-384 is identical to SHA-512, except that:

- the initial hash values h_0 through h_7 are different (taken from the 9th through 16th primes), and
- the output is constructed by omitting h_6 and h_7 .

SHA-384 initial hash values (in big-endian):

```

h[0..7] := 0xcbbb9d5dc1059ed8, 0x629a292a367cd507, 0x9159015a3070dd17, 0x152fecdf70e5939,
0x67332667ffc00b31, 0x8eb44a8768581511, 0xdb0c2e0d64f98fa7, 0x47b5481dbefa4fa4

```

SHA-512/t is identical to SHA-512 except that:

- the initial hash values h_0 through h_7 are given by the *SHA-512/t IV generation function*,
- the output is constructed by truncating the concatenation of h_0 through h_7 at t bits,
- t equal to 384 is not allowed, instead SHA-384 should be used as specified, and
- t values 224 and 256 are especially mentioned as approved.

SHA-512/224 initial hash values (in big-endian):

```

h[0..7] := 0x8c3d37c819544da2, 0x73e1996689dcd4d6, 0x1dfab7ae32ff9c82, 0x679dd514582f9fcf,
0x0f6d2b697bd44da8, 0x77e36f7304c48942, 0x3f9d85a86a1d36c8, 0x1112e6ad91d692a1

```

SHA-512/256 initial hash values (in big-endian):

```

h[0..7] := 0x22312194fc2bf72c, 0x9f555fa3c84c64c2, 0x2393b86b6f53b151, 0x963877195940eabd,
0x96283ee2a88effe3, 0xbe5e1e2553863992, 0x2b0199fc2c85b8aa, 0x0eb72ddc81c52ca2

```

The *SHA-512/t IV generation function* evaluates a *modified SHA-512* on the ASCII string "SHA-512/t", substituted with the decimal representation of *t*. The *modified SHA-512* is the same as SHA-512 except its initial values `h0` through `h7` have each been XORed with the hexadecimal constant `0xa5a5a5a5a5a5a5a5`.

Sample C implementation for SHA-2 family of hash functions can be found in RFC 6234 (<https://www.rfc-editor.org/rfc/rfc6234>).

Comparison of SHA functions

In the table below, *internal state* means the "internal hash sum" after each compression of a data block.

Comparison of SHA functions

Algorithm and variant		Output size (bits)	Internal state size (bits)	Block size (bits)	Rounds	Operations	Security (bits)	Performance on Skylake (median cpb) ^[47]		First published
								Long messages	8 bytes	
MD5 (as reference)		128	128 (4 × 32)	512	4 (16 operations in each round)	And, Xor, Or, Rot, Add (mod 2 ³²)	≤ 18 (collisions found) ^[48]	4.99	55.00	1992
SHA-0		160	160 (5 × 32)	512	80	And, Xor, Or, Rot, Add (mod 2 ³²)	< 34 (collisions found)	≈ SHA-1	≈ SHA-1	1993
SHA-1							< 63 (collisions found) ^[49]	3.47	52.00	1995
SHA-2	SHA-224 SHA-256	224 256	256 (8 × 32)	512	64	And, Xor, Or, Rot, Shr, Add (mod 2 ³²)	112 128	7.62 7.63	84.50 85.25	2004 2001
	SHA-384	384	512 (8 × 64)	1024	80	And, Xor, Or, Rot, Shr, Add (mod 2 ⁶⁴)	192	5.12	135.75	2001
	SHA-512	512					256	5.06	135.50	2001
	SHA-512/224 SHA-512/256	224 256					112 128	≈ SHA-384	≈ SHA-384	2012
SHA-3	SHA3-224 SHA3-256 SHA3-384 SHA3-512	224 256 384 512	1600 (5 × 5 × 64)	1152 1088 832 576	24 ^[50]	And, Xor, Rot, Not	112 128 192 256	8.12 8.59 11.06 15.88	154.25 155.50 164.00 164.00	2015
	SHAKE128 SHAKE256	<i>d</i> (arbitrary) <i>d</i> (arbitrary)		1344 1088			min(<i>d</i> /2, 128) min(<i>d</i> /2, 256)	7.08 8.59	155.25 155.50	

In the bitwise operations column, "Rot" stands for rotate no carry, and "Shr" stands for right logical shift. All of these algorithms employ modular addition in some fashion except for SHA-3.

More detailed performance measurements on modern processor architectures are given in the table below.

CPU architecture	Frequency	Algorithm	Word size (bits)	Cycles/byte <u>x86</u>	MiB/s <u>x86</u>	Cycles/byte <u>x86-64</u>	MiB/s <u>x86-64</u>
Intel Ivy Bridge	3.5 GHz	SHA-256	32	16.80	199	13.05	256
		SHA-512	64	43.66	76	8.48	394
AMD Piledriver APU	3.8 GHz	SHA-256	32	22.87	158	18.47	196
		SHA-512	64	88.36	41	12.43	292

The performance numbers labeled 'x86' were running using 32-bit code on 64-bit processors, whereas the 'x86-64' numbers are native 64-bit code. While SHA-256 is designed for 32-bit calculations, it does benefit from code optimized for 64-bit processors on the x86 architecture. 32-bit implementations of SHA-512 are significantly slower than their 64-bit counterparts. Variants of both algorithms with different output sizes will perform similarly, since the message expansion and compression functions are identical, and only the initial hash values and output sizes are different. The best implementations of MD5 and SHA-1 perform between 4.5 and 6 cycles per byte on modern processors.

Testing was performed by the University of Illinois at Chicago on their hydra8 system running an Intel Xeon E3-1275 V2 at a clock speed of 3.5 GHz, and on their hydra9 system running an AMD A10-5800K APU at a clock speed of 3.8 GHz.^[51] The referenced cycles per byte speeds above are the median performance of an algorithm digesting a 4,096 byte message using the SUPERCOP cryptographic benchmarking software.^[52] The MiB/s performance is extrapolated from the CPU clockspeed on a single core; real-world performance will vary due to a variety of factors.

SHA-256 and SHA-512 are vulnerable to a length extension attack, but SHA-384 and SHA-256/512 are not vulnerable because their output is a truncated version of their internal state.^[53]

Implementations

Cryptography libraries that support SHA-2:

- [Botan](#)
- [Bouncy Castle](#)
- [Cryptlib](#)
- [Crypto++](#)
- [Libgcrypt](#)
- [Mbed TLS](#)^{[54][55]}
- [libsodium](#)
- [Nettle](#)
- [LibreSSL](#)
- [OpenSSL](#)
- [GnuTLS](#)
- [wolfSSL](#)

Hardware acceleration is provided by the following processor extensions:

- [Intel SHA extensions](#): Available on some Intel and AMD x86 processors.
- [VIA PadLock](#)
- [ARMv8 Cryptography Extensions](#)^[56]
- [IBM z/Architecture](#): Available since 2005 as part of the Message-Security-Assist Extensions 1 (SHA-256) and 2 (SHA-512)^[57]
- [IBM Power ISA](#) since v.2.07

See also

- [Comparison of cryptographic hash functions](#)
- [Comparison of cryptography libraries](#)
- [Hash function security summary](#)
- [Hashcash](#)
- [HMAC](#)
- [International Association for Cryptologic Research \(IACR\)](#)
- [Trusted timestamping](#)

References

1. Khovratovich, Dmitry; Rechberger, Christian & Savelieva, Alexandra (2011). "Bicliques for Preimages: Attacks on Skein-512 and the SHA-2 family" (<https://eprint.iacr.org/2011/286.pdf>) (PDF). *IACR Cryptology ePrint Archive*. **2011** (286). Archived (<https://web.archive.org/web/20220215055932/https://eprint.iacr.org/2011/286.pdf>) (PDF) from the original on 2022-02-15. Retrieved 2022-02-15.
2. Lamberger, Mario & Mendel, Florian (2011). "Higher-Order Differential Attack on Reduced SHA-256" (<https://eprint.iacr.org/2011/037.pdf>) (PDF). *IACR Cryptology ePrint Archive*. **2011** (37). Archived (<https://web.archive.org/web/20221222014546/https://eprint.iacr.org/2011/037.pdf>) (PDF) from the original on 2022-12-22. Retrieved 2022-02-15.
3. Penard, Wouter; van Werkhoven, Tim. "On the Secure Hash Algorithm family" (https://web.archive.org/web/20160330153520/https://www.staff.science.uu.nl/~werkh108/docs/study/Y5_07_08/infocry/project/Cryp08.pdf) (PDF). *staff.science.uu.nl*. Archived from the original (https://www.staff.science.uu.nl/~werkh108/docs/study/Y5_07_08/infocry/project/Cryp08.pdf) (PDF) on 2016-03-30.
4. Federal Register Notice 02-21599, Announcing Approval of FIPS Publication 180-2 (<https://federalregister.gov/a/02-21599>) Archived (<https://web.archive.org/web/20220314024321/https://www.federalregister.gov/a/02-21599>) 2022-03-14 at the [Wayback Machine](#)
5. "IPR Details: The United States of America as represented by the National Security Agency's general license statement" (<https://datatracker.ietf.org/ipr/858/>). *IETF Datatracker*. 858. Archived (<https://web.archive.org/web/20160616212010/https://datatracker.ietf.org/ipr/858/>) from the original on 2016-06-16. Retrieved 2008-02-17.
6. US 6829355 (<https://worldwide.espacenet.com/textdoc?DB=EPODOC&IDX=US6829355>), Lilly, Glenn M., "Device for and method of one-way cryptographic hashing", published 2004-12-07, assigned to [National Security Agency](#)
7. Algazy et al. 2024.
8. "FIPS 180-2 with Change Notice 1" (<https://csrc.nist.gov/publications/fips/fips180-2/fips180-2withchangenotice.pdf>) (PDF). *csrc.nist.gov*. Archived (<https://web.archive.org/web/20170809001643/http://csrc.nist.gov/publications/fips/fips180-2/fips180-2withchangenotice.pdf>) (PDF) from the original on 2017-08-09. Retrieved 2022-02-15.
9. Federal Register Notice E8-24743, Announcing Approval of FIPS Publication 180-3 (<https://federalregister.gov/a/E8-24743>)
10. Dang, Quynh (2012-08-24). Recommendation for Applications Using Approved Hash Algorithms (<https://csrc.nist.gov/Pubs/sp/800/107/r1/Final>) (Report). National Institute of Standards and Technology. Archived (<https://web.archive.org/web/20230828000235/https://csrc.nist.gov/Pubs/sp/800/107/r1/Final>) from the original on 2023-08-28. Retrieved 2023-08-28.

11. Barker, Elaine; Barker, William; Burr, William; Polk, W.; Smid, Miles (2012-07-10). Recommendation for Key Management, Part 1: General (Revision 3) (<https://csrc.nist.gov/Pubs/sp/800/57/pt1/r3/Final>) (Report). National Institute of Standards and Technology. Archived (<https://web.archive.org/web/20230828000234/https://csrc.nist.gov/Pubs/sp/800/57/pt1/r3/Final>) from the original on 2023-08-28. Retrieved 2023-08-28.
12. "NIST.gov – Computer Security Division – Computer Security Resource Center" (<https://csrc.nist.gov/groups/ST/toolkit/examples.html#aHashing>). 29 December 2016. Archived (<https://web.archive.org/web/20170909052333/http://csrc.nist.gov/groups/ST/toolkit/examples.html#aHashing>) from the original on 9 September 2017. Retrieved 15 February 2022.
13. Barker, Elaine; Roginsky, Allen (2011-01-13). Transitions: Recommendation for Transitioning the Use of Cryptographic Algorithms and Key Lengths (<https://csrc.nist.gov/Pubs/sp/800/131/a/Final>) (Report). National Institute of Standards and Technology. Archived (<https://web.archive.org/web/20230828000236/https://csrc.nist.gov/Pubs/sp/800/131/a/Final>) from the original on 2023-08-28. Retrieved 2023-08-28.
14. Federal Register Notice 2012-5400, Announcing Approval of FIPS Publication 180-4 (<https://federalregister.gov/a/2012-5400>).
15. NIST, Decision to Revise FIPS 180-4, Secure Hash Standard (SHS) (<https://csrc.nist.gov/news/2023/decision-to-revise-fips-180-4>)
16. "NIST Selects Winner of Secure Hash Algorithm (SHA-3) Competition" (<https://www.nist.gov/itl/csd/sha-100212.cfm>). NIST. 2 October 2012. Archived (<https://web.archive.org/web/20150402081721/http://www.nist.gov/itl/csd/sha-100212.cfm>) from the original on 2 April 2015. Retrieved 24 February 2015.
17. Franck, Lucas Daudt; Ginja, Gabriel Augusto; Carmo, João Paulo; Afonso, Jose A.; Luppe, Maximilian (2024). "Custom ASIC Design for SHA-256 Using Open-Source Tools" (<https://doi.org/10.3390/2Fcomputers13010009>). *Computers*. **13** (1): 9. doi:10.3390/computers13010009 (<https://doi.org/10.3390/2Fcomputers13010009>). hdl:1822/89307 (<https://hdl.handle.net/1822/2F89307>).
18. "Verifying authenticity of Debian images" (<https://www.debian.org/CD/verify>). Archived (<https://web.archive.org/web/20240219133631/https://www.debian.org/CD/verify>) from the original on 2024-02-19. Retrieved 2024-02-19.
19. Markoff, John (2009-01-27). "A Tool to Verify Digital Records, Even as Technology Shifts" (<https://www.nytimes.com/2009/01/27/science/27arch.html>). *The New York Times*. ISSN 0362-4331 (<https://search.worldcat.org/issn/0362-4331>). Archived (<https://web.archive.org/web/20230919011821/https://www.nytimes.com/2009/01/27/science/27arch.html>) from the original on 2023-09-19. Retrieved 2023-08-27.
20. Hardaker, Wes (2022-08-12). Remove SHA-1 from active use within DNSSEC (<https://www.ietf.org/archive/id/draft-hardaker-dnsop-must-not-sha1-00.html>) (Report). Internet Engineering Task Force.
21. "Security/Features - Debian Wiki" (<https://wiki.debian.org/Security/Features>). *wiki.debian.org*. Retrieved 2025-01-13.
22. "SHA hashes – Arch Wiki" (https://wiki.archlinux.org/title/SHA_hashes). *wiki.archlinux.org*. Retrieved 2025-01-13.
23. "Bitcoin Does Not Waste Energy" (<https://web.archive.org/web/20220528202245/https://surplusbitcoin.com/>). *Surplus Bitcoin*. Archived from the original (<https://surplusbitcoin.com/>) on 2022-05-28. Retrieved 2020-04-20.
24. "What Is SHA-256 And How Is It Related to Bitcoin? - Mycryptopedia" (<https://www.mycryptopedia.com/sha-256-related-bitcoin/>). *Mycryptopedia*. 2017-09-21. Archived (<https://web.archive.org/web/20180917215259/https://www.mycryptopedia.com/sha-256-related-bitcoin/>) from the original on 2018-09-17. Retrieved 2018-09-17.
25. 3GPP TS 33.401, Security architecture and procedures for E-UTRAN (<https://portal.3gpp.org/desktopmodules/Specifications/SpecificationDetails.aspx?specificationId=2296>)
26. 3GPP TS 33.501, Security architecture and procedures for 5G systems (<https://portal.3gpp.org/desktopmodules/Specifications/SpecificationDetails.aspx?specificationId=3169>)
27. Computer Security Division, Information Technology Laboratory (2017-01-04). "NIST Policy on Hash Functions – Hash Functions | CSRC | NIST" (<https://csrc.nist.gov/projects/hash-functions/nist-policy-on-hash-functions>). CSRC | NIST. Archived (<https://web.archive.org/web/20230828000235/https://csrc.nist.gov/projects/hash-functions/nist-policy-on-hash-functions>) from the original on 2023-08-28. Retrieved 2023-08-27.
28. "Secure Hashing" (https://web.archive.org/web/20110625054822/https://csrc.nist.gov/groups/ST/toolkit/secure_hashing.html). NIST. Archived from the original (https://csrc.nist.gov/groups/ST/toolkit/secure_hashing.html) on 2011-06-25. Retrieved 2010-11-25.
29. "Overview of Windows XP Service Pack 3" (<https://web.archive.org/web/20080530175317/https://download.microsoft.com/download/6/8/7/687484ed-8174-496d-8db9-f02b40c12982/Overview%20of%20Windows%20XP%20Service%20Pack%203.pdf>) (PDF). Microsoft Corporation. Archived from the original (<https://download.microsoft.com/download/6/8/7/687484ed-8174-496d-8db9-f02b40c12982/Overview%20of%20Windows%20XP%20Service%20Pack%203.pdf>) (PDF) on May 30, 2008.
30. "Gradually Sunseting SHA-1" (<https://blog.chromium.org/2014/09/gradually-sunsetting-sha-1.html>). *Chromium Blog*. Archived (<https://web.archive.org/web/20230807123806/https://blog.chromium.org/2014/09/gradually-sunsetting-sha-1.html>) from the original on 2023-08-07. Retrieved 2023-08-27.
31. Mill, Eric. "SHAAAAAAAAAAAAAAAA" (<https://shaaaaaaaaaaaaa.com/>). SHAAAAAAAAAAAAAAAA.com. Archived (<https://web.archive.org/web/20170301035323/https://shaaaaaaaaaaaaa.com/>) from the original on 2017-03-01. Retrieved 2015-08-26.
32. "The unofficial Chrome SHA1 deprecation FAQ" (<https://words.filippo.io/the-unofficial-chrome-sha1-faq/>). *Filippo Valsorda*. 2015-04-08. Archived (<https://web.archive.org/web/20230828000235/https://words.filippo.io/the-unofficial-chrome-sha1-faq/>) from the original on 2023-08-28. Retrieved 2023-08-27.
33. "An update to our SHA-1 deprecation roadmap – Microsoft Edge Dev Blog" (<https://blogs.windows.com/msedgedev/2016/04/29/sha1-deprecation-roadmap>). *blogs.windows.com*. 29 April 2016. Archived (<https://web.archive.org/web/20161128200047/https://blogs.windows.com/msedgedev/2016/04/29/sha1-deprecation-roadmap/>) from the original on 2016-11-28. Retrieved 2016-11-28.
34. "Firefox: Mozilla schaltet SHA-1 ab ... und direkt wieder an" (<https://www.heise.de/news/Firefox-Mozilla-schaltet-SHA-1-ab-und-direkt-wieder-an-3066832.html>). *heise.de* (in German). 2016-01-08. Archived (<https://web.archive.org/web/20230828000234/https://www.heise.de/news/Firefox-Mozilla-schaltet-SHA-1-ab-und-direkt-wieder-an-3066832.html>) from the original on 2023-08-28. Retrieved 2025-01-18.

35. Alexander Sotirov, Marc Stevens, Jacob Appelbaum, Arjen Lenstra, David Molnar, Dag Arne Osvik, Benne de Weger, MD5 considered harmful today: Creating a rogue CA certificate (<https://www.win.tue.nl/hashclash/rogue-ca/>). Archived (<https://web.archive.org/web/20220323052759/https://www.win.tue.nl/hashclash/rogue-ca/>) 2022-03-23 at the Wayback Machine, accessed March 29, 2009.
36. Ji Li, Takanori Isobe and Kyoji Shibutani, Sony China Research Laboratory and Sony Corporation, Converting Meet-in-the-Middle Preimage Attack into Pseudo Collision Attack: Application to SHA-2 (<https://fse2012.inria.fr/SLIDE/S/67.pdf>). Archived (<https://web.archive.org/web/20220224204936/http://fse2012.inria.fr/SLIDES/67.pdf>) 2022-02-24 at the Wayback Machine.
37. Sanadhya, Somitra Kumar; Sarkar, Palash (2008), *New collision attacks against up to 24-step SHA-2* (https://link.springer.com/chapter/10.1007/978-3-540-89754-5_8), Lecture Notes in Computer Science, vol. 5365, Springer-Verlag, pp. 91–103, doi:10.1007/978-3-540-89754-5_8 (https://doi.org/10.1007%2F978-3-540-89754-5_8), ISBN 978-3-540-89753-8, archived (https://web.archive.org/web/20220121011031/https://link.springer.com/chapter/10.1007%2F978-3-540-89754-5_8) from the original on 2022-01-21, retrieved 2024-02-12.
38. Sanadhya, Somitra Kumar; Sarkar, Palash (2009). "A combinatorial analysis of recent attacks on step reduced SHA-2 family" (<https://link.springer.com/article/10.1007/s12095-009-0011-5>). *Cryptography and Communications*. **1** (2): 135–173. doi:10.1007/s12095-009-0011-5 (https://doi.org/10.1007%2F978-3-540-89754-5_8). Archived (<https://web.archive.org/web/20230802133147/https://link.springer.com/article/10.1007/s12095-009-0011-5>) from the original on 2023-08-02. Retrieved 2024-02-12.
39. Aoki, Kazumaro; Guo, Jian; Matusiewicz, Krystian; Sasaki, Yu & Wang, Lei (2009). "Preimages for Step-Reduced SHA-2". *Advances in Cryptology – ASIACRYPT 2009*. Lecture Notes in Computer Science. Vol. 5912. Springer Berlin Heidelberg. pp. 578–597. doi:10.1007/978-3-642-10366-7_34 (https://doi.org/10.1007%2F978-3-642-10366-7_34). ISBN 978-3-642-10366-7. ISSN 0302-9743 (<https://search.worldcat.org/issn/0302-9743>).
40. Guo, Jian; Ling, San; Rechberger, Christian & Wang, Huaxiong (2010). "Advanced Meet-in-the-Middle Preimage Attacks: First Results on Full Tiger, and Improved Results on MD4 and SHA-2". *Advances in Cryptology – ASIACRYPT 2010* (<https://eprint.iacr.org/2010/016.pdf>) (PDF). Lecture Notes in Computer Science. Vol. 6477. Springer Berlin Heidelberg. pp. 56–75. doi:10.1007/978-3-642-17373-8_4 (https://doi.org/10.1007%2F978-3-642-17373-8_4). ISBN 978-3-642-17373-8. ISSN 0302-9743 (<https://search.worldcat.org/issn/0302-9743>). Archived (<https://web.archive.org/web/20220303171536/https://eprint.iacr.org/2010/016.pdf>) (PDF) from the original on 2022-03-03. Retrieved 2022-02-15.
41. Mendel, Florian; Nad, Tomislav; Schl  ffer, Martin (2013). "Improving Local Collisions: New Attacks on Reduced SHA-256". *Advances in Cryptology – EUROCRYPT 2013* (https://online.tugraz.at/tug_online/voe_main2.getvolltext?pCurrPk=69018). Lecture Notes in Computer Science. Vol. 7881. Springer Berlin Heidelberg. pp. 262–278. doi:10.1007/978-3-642-38348-9_16 (https://doi.org/10.1007%2F978-3-642-38348-9_16). ISBN 978-3-642-38348-9. ISSN 0302-9743 (<https://search.worldcat.org/issn/0302-9743>). Archived (https://web.archive.org/web/20181106192811/https://online.tugraz.at/tug_online/voe_main2.getvolltext?pCurrPk=69018) from the original on 2018-11-06. Retrieved 2014-12-13.
42. Eichlseder, Maria; Mendel, Florian; and Schl  ffer, Martin (2014). "Branching Heuristics in Differential Collision Search with Applications to SHA-512" (<https://eprint.iacr.org/2014/302.pdf>) (PDF). *IACR Cryptology ePrint Archive*. **2014** (302). Archived (<https://web.archive.org/web/20220120220202/https://eprint.iacr.org/2014/302.pdf>) (PDF) from the original on 2022-01-20. Retrieved 2022-02-15.
43. Dobraunig, Christoph; Eichlseder, Maria & Mendel, Florian (2016). "Analysis of SHA-512/224 and SHA-512/256" (<https://eprint.iacr.org/2016/374.pdf>) (PDF). *International Association for Cryptologic Research*. Archived (<https://web.archive.org/web/20170715223048/https://eprint.iacr.org/2016/374.pdf>) (PDF) from the original on 2017-07-15. Retrieved 2016-04-15.
44. Li, Yingxin; Liu, Fukang; Wang, Gaoli (2024). "New Records in Collision Attacks on SHA-2" (<https://eprint.iacr.org/2024/349>). *Cryptology ePrint Archive*. Archived (<https://web.archive.org/web/20240302224244/https://eprint.iacr.org/2024/349>) from the original on 2024-03-02. Retrieved 2024-03-02.
45. "Secure Hashing – Cryptographic Algorithm Validation Program" (<https://csrc.nist.gov/projects/cryptographic-algorithm-validation-program/secure-hashing>). *NIST CSRC*. 5 October 2016. Retrieved 8 November 2025.
46. "SHS Validation List" (<https://web.archive.org/web/20170617035122/https://csrc.nist.gov/groups/STM/cavp/documents/shs/shaval.html>). *NIST*. 2017-06-16. Archived from the original (<https://csrc.nist.gov/groups/STM/cavp/documents/shs/shaval.html>) on 2017-06-17.
47. "Measurements table" (<http://bench.cr.yp.to/results-hash.html#amd64-skylake>). *bench.cr.yp.to*.
48. Tao, Xie; Liu, Fanbao; Feng, Dengguo (2013). *Fast Collision Attack on MD5* (<https://eprint.iacr.org/2013/170.pdf>) (PDF). *Cryptology ePrint Archive* (Technical report). IACR.
49. Stevens, Marc; Bursztein, Elie; Karpman, Pierre; Albertini, Ange; Markov, Yarik. *The first collision for full SHA-1* (<https://web.archive.org/web/20260207153244/https://shattered.io/static/shattered.pdf>) (PDF) (Technical report). Google Research. Archived from the original (<https://shattered.io/static/shattered.pdf>) (PDF) on 7 February 2026.
 - Marc Stevens; Elie Bursztein; Pierre Karpman; Ange Albertini; Yarik Markov; Alex Petit Bianco; Clement Baisse (February 23, 2017). "Announcing the first SHA1 collision" (<https://security.googleblog.com/2017/02/announcing-first-sha1-collision.html>). *Google Security Blog*.
50. "The Keccak sponge function family" (http://keccak.noekion.org/specs_summary.html). Retrieved 2016-01-27.
51. SUPERCOP Benchmarks Measurements of hash functions, indexed by machine (<https://bench.cr.yp.to/results-hash.html>).
52. "SUPERCOP" (<https://bench.cr.yp.to/supercop.html>). Archived (<https://web.archive.org/web/20150215055126/http://bench.cr.yp.to/supercop.html>) from the original on 15 February 2015. Retrieved 24 February 2015.
53. Bostrom, Michael (2015-10-29). "size_t Does Matter: Hash Length Extension Attacks Explained" (https://bostik.iki.fi/dc4420/size_t-does-matter--hash-length-extensions.pdf) (PDF). Retrieved 2026-06-26.
54. "Supported SSL / TLS ciphersuites" (<https://tls.mbed.org/supported-ssl-ciphersuites>). Archived (<https://web.archive.org/web/20190512061037/https://tls.mbed.org/supported-ssl-ciphersuites>) from the original on 2019-05-12. Retrieved 2019-10-19.
55. "Mbed TLS Changelog, 7 July 2007" (<https://github.com/ARMmbed/mbedtls/blob/master/ChangeLog>). *GitHub*. Archived (<https://web.archive.org/web/20190204101359/https://github.com/ARMmbed/mbedtls/blob/master/ChangeLog>) from the original on 4 February 2019. Retrieved 19 October 2019.

56. "ARM Cortex-A53 MPCore Processor Technical Reference Manual Cryptography Extension" (<https://infocenter.arm.com/help/index.jsp?topic=/com.arm.doc.ddi0500e/CJHDEBAF.html>). Archived (<https://web.archive.org/web/20200601095542/http://infocenter.arm.com/help/index.jsp?topic=/com.arm.doc.ddi0500e/CJHDEBAF.html>) from the original on 2020-06-01. Retrieved 2022-02-15.
57. IBM z/Architecture Principles of Operation, publication number SA22-7832. See KIMD and KLMD instructions in Chapter 7.

Further reading

- Algazy, K.; Sakan, K.; Nyssanbayeva, S.; Lizunov, O. (2024). "Syrga2: Post-quantum hash-based signature scheme" (<https://doi.org/10.3390%2Fcomputation12060125>). *Computation*. **12** (6): 125. doi:10.3390/computation12060125 (<https://doi.org/10.3390%2Fcomputation12060125>).
- Henri Gilbert, Helena Handschuh: Security Analysis of SHA-256 and Sisters. *Selected Areas in Cryptography 2003*: pp. 175–193.
- "Proposed Revision of Federal Information Processing Standard (FIPS) 180, Secure Hash Standard" (<https://www.federalregister.gov/documents/1994/07/11/94-16666/proposed-revision-of-federal-information-processing-standard-fips-180-secure-hash-standard>). *Federal Register*. **59** (131): 35317–35318. 1994-07-11. Archived (<https://web.archive.org/web/20200728142609/https://www.federalregister.gov/documents/1994/07/11/94-16666/proposed-revision-of-federal-information-processing-standard-fips-180-secure-hash-standard>) from the original on 2020-07-28. Retrieved 2007-04-26.

External links

- Descriptions of SHA-256, SHA-384, and SHA-512 (<https://web.archive.org/web/20130526224224/https://csrc.nist.gov/groups/STM/cavp/documents/shs/sha256-384-512.pdf>) from NIST
- SHA-2 Checker (<https://shachecker.com>) – SHAChecker to check one's SSL compatibility for SHA-2
- SHA-256 Calculator (<https://fe-tool.com/hash/sha256>) – SHA-256 Calculator
- Specifications for a Secure Hash Standard (SHS) (https://web.archive.org/web/20141008212020/https://w2.eff.org/Privacy/Digital_signature/?f=fips_sha_shs.standard.txt) – Draft for proposed SHS (SHA-0)
- Secure Hash Standard (SHS) (https://web.archive.org/web/20141008212429/https://w2.eff.org/Privacy/Digital_signature/?f=fips_sha_shs.info.txt) – Proposed SHS (SHA-0)
- CSRC Cryptographic Toolkit (https://web.archive.org/web/20110625054822/https://csrc.nist.gov/groups/ST/toolkit/secure_hashing.html) – Official NIST site for the Secure Hash Standard
- FIPS PUB 180-4: Secure Hash Standard (SHS) (<https://web.archive.org/web/20161126003357/https://nvlpubs.nist.gov/nistpubs/FIPS/NIST.FIPS.180-4.pdf>) (PDF, 834 KB) – Current version of the Secure Hash Standard (SHA-1, SHA-224, SHA-256, SHA-384, and SHA-512), August 2015
- Test vectors for SHA-256/384/512 (<https://www.cosic.esat.kuleuven.be/nessie/testvectors/hash/sha/index.html>) from the NESSIE project
- Test vectors for SHA-1, SHA-2 (<https://csrc.nist.gov/groups/STM/cavp/index.html#03>) from NIST site
- NIST Cryptographic Hash Project (<https://web.archive.org/web/20100505162618/https://csrc.nist.gov/groups/ST/hash/index.html>) – SHA-3 competition
- RFC 3874 (<https://www.rfc-editor.org/rfc/rfc3874>): "A 224-bit One-way Hash Function: SHA-224"
- RFC 6234 (<https://www.rfc-editor.org/rfc/rfc6234>): "US Secure Hash Algorithms (SHA and SHA-based HMAC and HKDF)"; contains sample C implementation
- SHA-256 algorithm demonstration (<https://sha256algorithm.com/>)

Retrieved from "<https://en.wikipedia.org/w/index.php?title=SHA-2&oldid=1361621484>"